

CS:5980

Foundations of Embedded Systems

Safety Requirements

Part I

Copyright 2014-26, Rajeev Alur and Cesare Tinelli.

Created by Cesare Tinelli at the University of Iowa from notes originally developed by Rajeev Alur at the University of Pennsylvania. These notes are copyrighted materials and may not be used in other course settings outside of the University of Iowa in their current form or modified form without the express written permission of one of the copyright holders. During this course, students are prohibited from selling notes to or being paid for taking notes by any person or commercial firm without the express written permission of one of the copyright holders.

Requirements

Desirable **properties** of the executions of the system

- **Informal:** either implicit, or stated in natural language
- **Formal:** stated explicitly in a mathematically precise way

Model/design/system **meets** the requirements if **every** execution satisfies them

Clear separation between

- **requirements**, **what** needs to be implemented, and
- **system**, **how** it is implemented

Requirements

High-assurance/safety-critical systems are typically provided with precise requirements

Verification Problem:

Given a formally specified requirement **R** and a system/model **C**,
prove or **disprove** that **C** satisfies **R**

Safety and Liveness Requirements

Requirements can be broadly divided into two classes:

1. *Safety requirements* state that a system always stays within *good* states (i.e., *nothing bad ever happens*)
 - **Leader election**: it is *never* the case that two nodes consider themselves to be leaders
 - **Collision avoidance**: Distance between two cars is *always* greater than some minimum threshold
2. *Liveness requirements* state that a system eventually achieves its goal (i.e., *something good eventually happens*)
 - **Leader election**: Each node *eventually* makes a decision
 - **Cruise controller**: Actual speed *eventually* equals desired speed

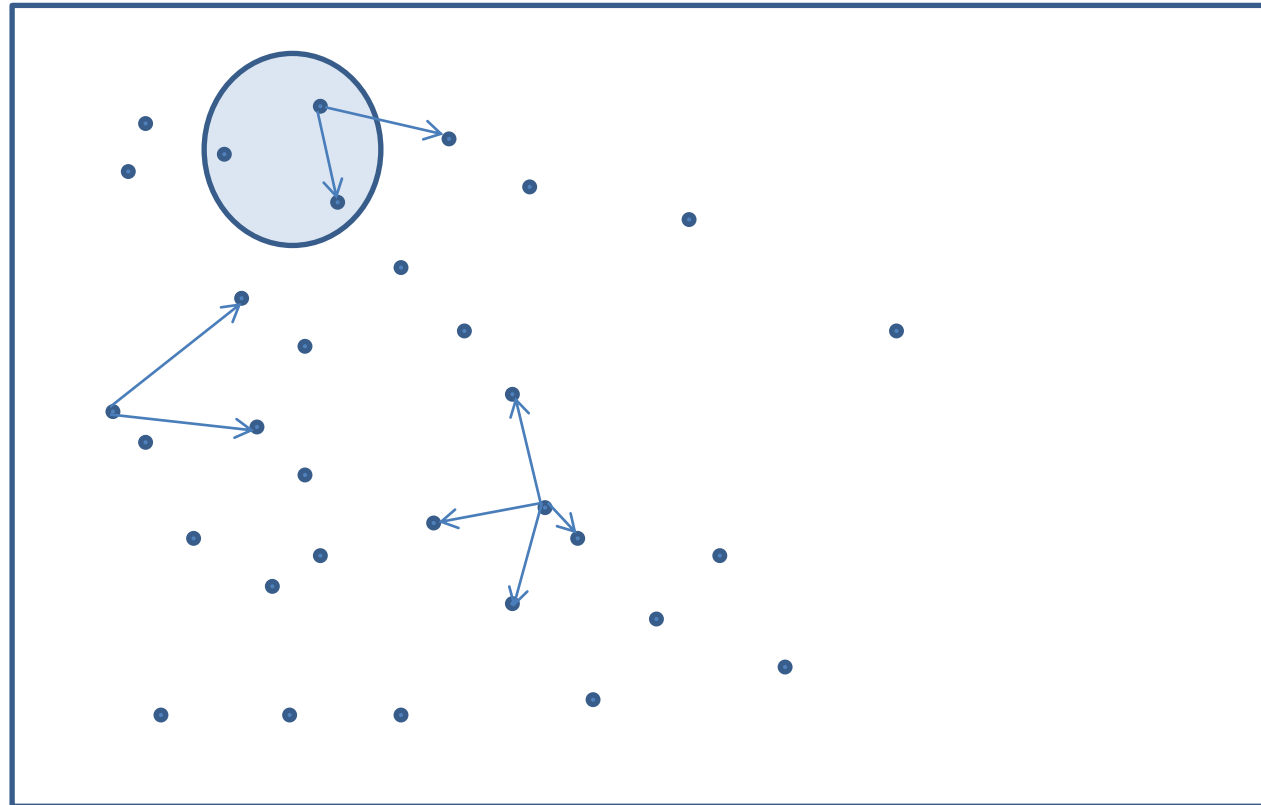
Safety and Liveness Requirements

Formalization and analysis techniques for **safety** and **liveness** differ significantly

We will start with safety

Transition Systems

State space + Initial states + Transitions between states



Definition of Transition System

Syntax: a transition system T has

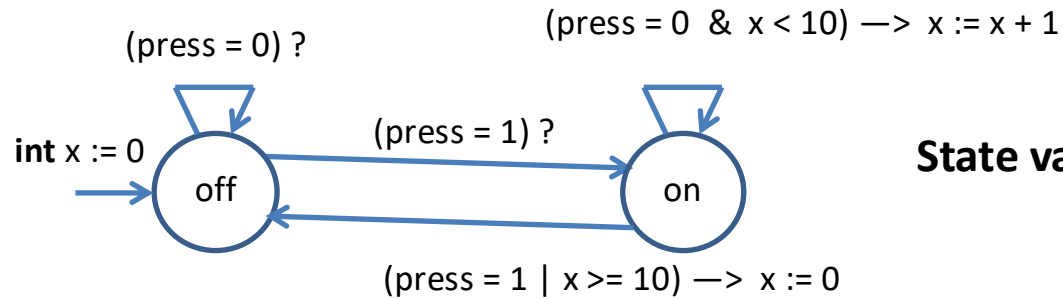
1. A set S of (typed) state variables
2. An initialization $Init$ for state variables
3. A description $Trans$ of how to move from one state to the next

Semantics:

1. S ranges over a set Q_S of states
2. $Init$ denotes a set $\llbracket Init \rrbracket$ of initial states, a subset of Q_S
3. $Trans$ denotes a set $\llbracket Trans \rrbracket$ of transitions, a subset of $Q_S \times Q_S$

Synchronous reactive components, EMS, programs, and computational systems in general,
all have an underlying transition system

Switch Transition System

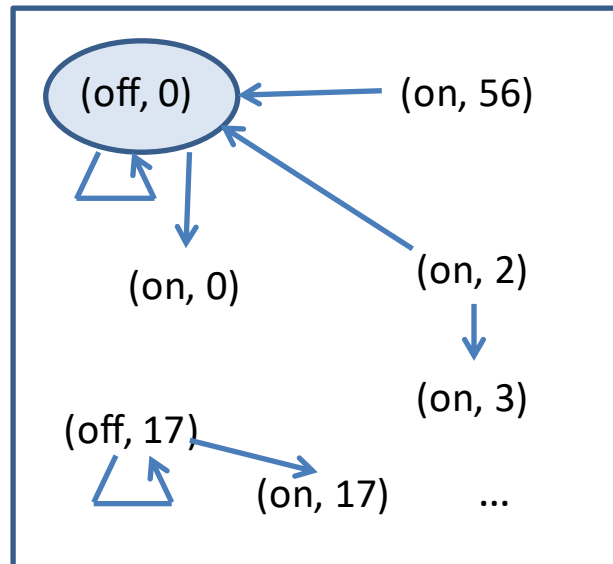


State variables:

`{off, on} mode, int x`

Initialization:

`mode := off ; x := 0`



Transitions:

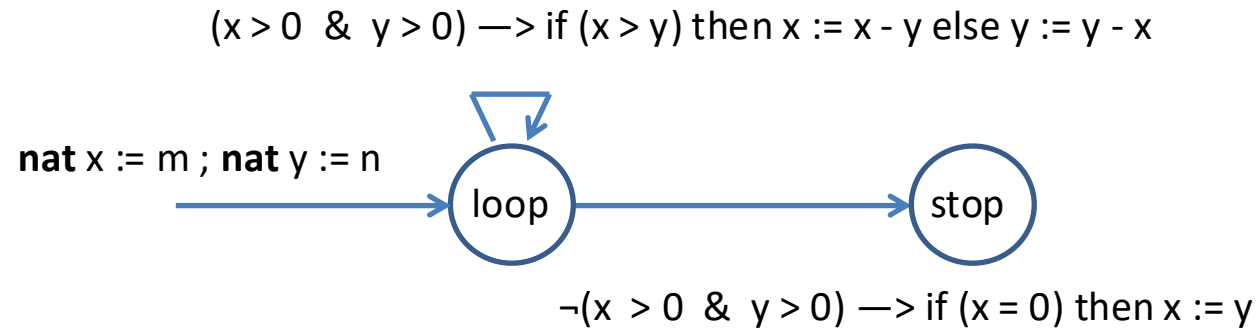
`(off, n) -> (off, n)`
`(off, n) -> (on, n)`
`(on, n) -> (on, n+1)`
`(on, n) -> (off, 0)`

for all n
 for all n
 for all n < 10
 for all n

- Input/output variables become **local vars**
- Values for input vars are chosen **non-deterministically**

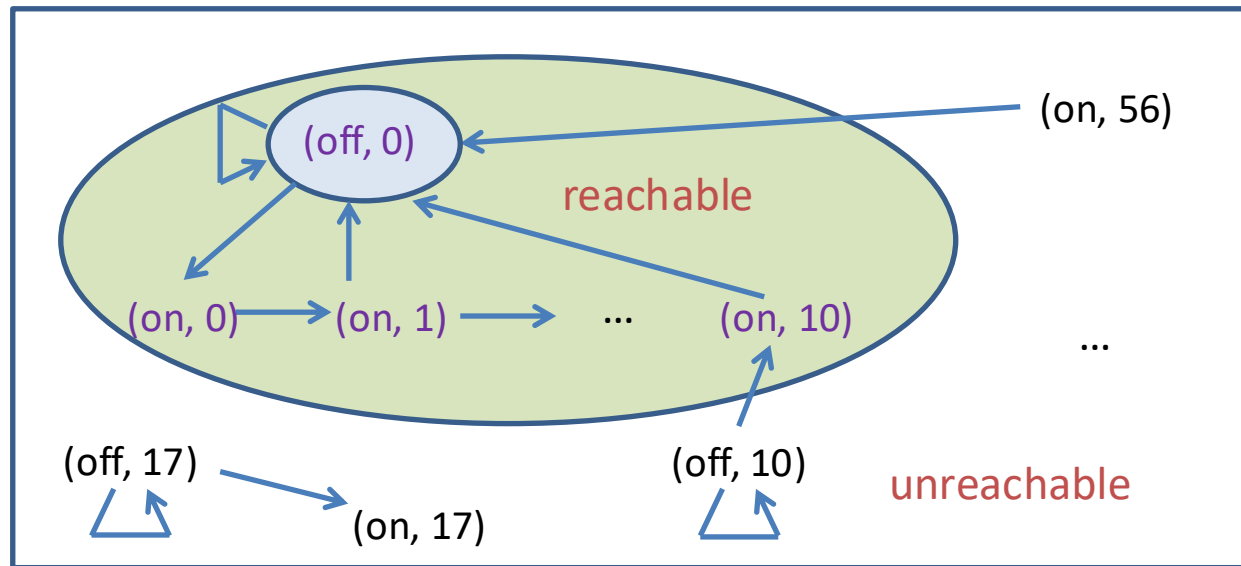
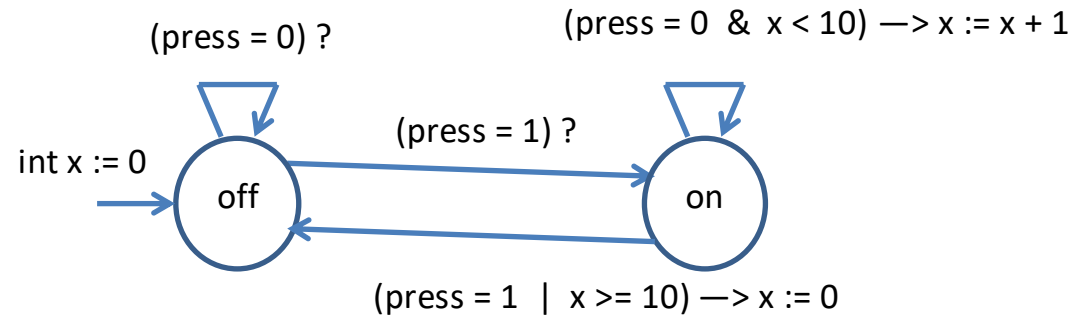
Euclid's GCD Algorithm

Classical program to compute greatest common divisor of (non-negative) input numbers m and n

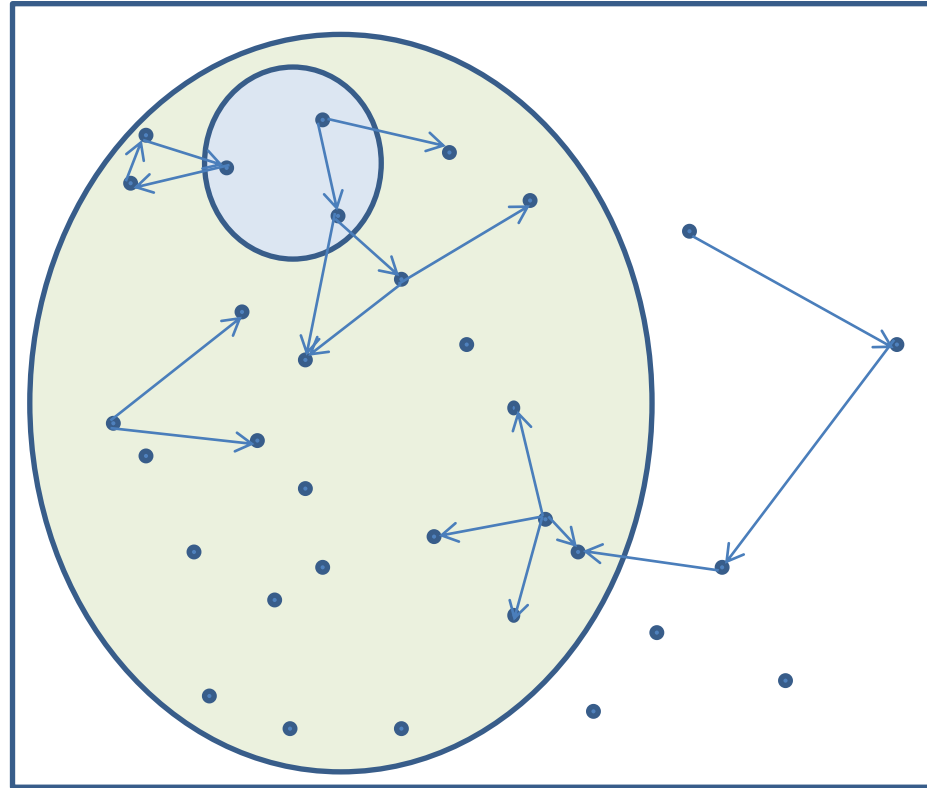


Reachable States

Switch

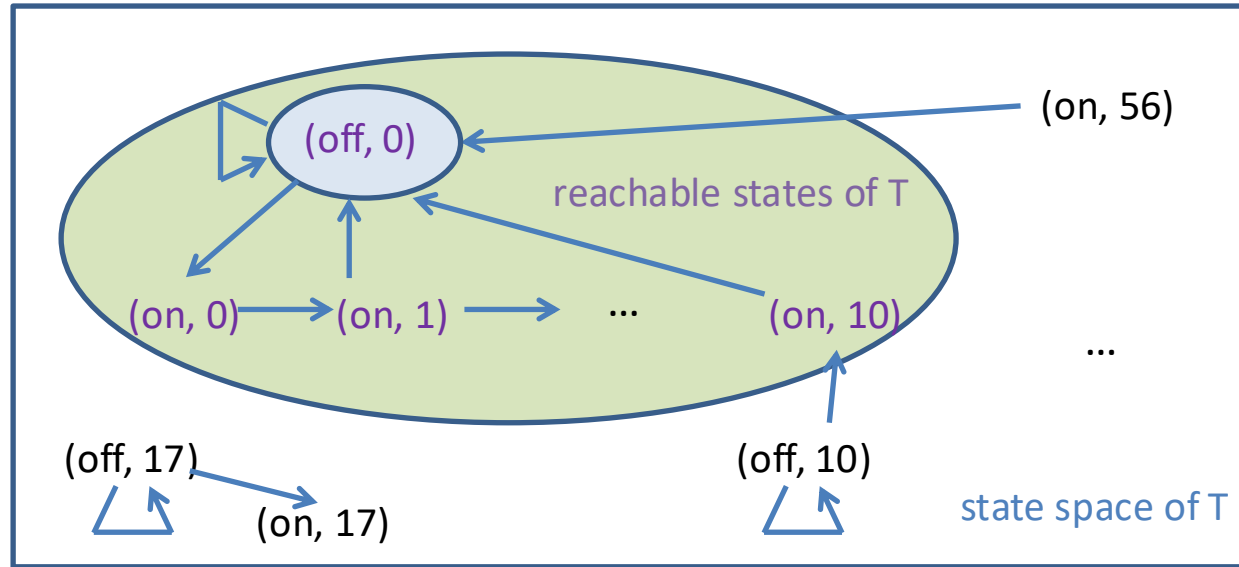


Reachable States of Transition Systems



A state s of a transition system T is *reachable* if there is an execution starting in an initial state of T and ending in s

Invariants



- A *property* of a transition system T is a Boolean expression P over state variables
- Property P is an *invariant* of T if **every reachable state** satisfies P
- Some invariants of T above: $x \leq 10$, $x \leq 50$, $mode = off \Rightarrow x = 0$
- Some non-invariants of T : $x < 10$, $mode = off$

Invariants

We express safety requirements for a transition system T as properties P of T 's state variables

- If P is invariant, then T is *safe* (wrt P)
- If P is not invariant, then some *bad* state, satisfying $\neg P$, is reachable (the execution leading to such a state is a *counterexample*)

Examples:

Leader election: $(r_n = N) \Rightarrow (id_n = \max I)$ I = set of identifiers of all nodes

Euclid's GCD Program: $(mode = stop) \Rightarrow (x = \gcd(m, n))$

Formal Verification



Grand challenge: automate verification as much as possible!

Analysis Techniques

Dynamic Analysis (runtime)

- Execute the system, possibly multiple times with different inputs
- Check if every execution meets the desired requirement

Static Analysis (design time)

- Analyze the source code or the model for possible bugs

Trade-offs:

- Dynamic analysis is **incomplete** but **accurate**
(checks real system, and discovered bugs are real bugs)
- Static analysis can be complete and can **catch design bugs early** but many static analysis techniques are not scalable
(solution: analyze approximate models, which can lead to false warnings)

Invariant Verification

Simulation

- Simulate the model, possibly multiple times with different inputs
- Easy to implement, scalable, but no correctness guarantees

Deductive verification

- Construct a proof that system satisfies the invariant
- Usually requires manual effort (but partial automation often possible)

Model checking

- Automatically explores all reachable states to check invariants
- Not scalable, but current tools can analyze many real-world designs (relies on many interesting theoretical advances)

Note: More recent techniques have blurred the differences between deductive verification and model checking

Proving Invariants

Given a transition system $T = (S, \text{Init}, \text{Trans})$, and a property P , prove that P is invariant for T , i.e., all reachable states of T satisfy P

Inductive definition of *reachable state*:

- All initial states are reachable in 0 transitions
- If a state s is reachable in k transitions and $s \longrightarrow t$ is a transition, then state t is reachable in $k+1$ transitions
- Reachable = Reachable in n transitions, for some n

Prove: for all n , every state reachable in n transitions satisfies P

Base case: Show that all initial states satisfy P

Inductive step:

1. Assume that a state s satisfies P
2. Show that if $s \longrightarrow t$ is a transition then t must satisfy P

Recall: Inductive Proofs in Arithmetic

To show that a statement P holds for all natural numbers n ,

- Base case: Prove that P holds for $n = 0$
- Assume that P holds for an arbitrary natural k
- Using the assumption, prove that P holds for $k + 1$

Example: For all naturals n , $\sum_{i=0}^n i = \frac{n(n+1)}{2}$

Inductive Invariant

A property P is an *inductive invariant* of transition system T if

1. Every initial state of T satisfies P
2. If a state satisfies P and $s \rightarrow t$ is a transition of T , then t satisfies P

Note:

1. If P is an *inductive invariant* of T , then all reachable states of T must satisfy P , and thus, it *is an invariant* of T
2. There are invariants that are *not* inductive

Proving Inductive Invariants: Example 1

Consider a transition system T given by

- State variable $\text{int } x$, initialized to 0
- Transition description given by $\text{if } (x < m) \text{ then } x := x + 1$ for some $m \geq 0$

Is the property $P: 0 \leq x \leq m$ an inductive invariant of T ?

Base case: Consider initial state $x := 0$. Check that it satisfies P

Inductive step:

- Consider an arbitrary state s , suppose $s(x) = a$
- Assume s satisfies P , that is, assume $0 \leq a \leq m$
- Consider the state t obtained by executing a transition from s
- If $a < m$ then $t(x) = a + 1$, else $t(x) = a$
- In either case, $0 \leq t(x) \leq m$
- So, t satisfies the property P , and the proof is complete

Proving Inductive Invariants: Example 2

Consider a transition system T given by

- State variables $\text{int } x, y$; initially: $x := 0 ; y := m$ for some $m > 0$
- Transition description given by $\text{if } (x < m) \text{ then } \{ x := x + 1 ; y := y - 1 \}$

Is the property $P : 0 \leq y \leq m$ an inductive invariant of T ?

Base case: Consider initial state $(x, y) := (0, m)$. Check that it satisfies T

Inductive step:

- Consider an arbitrary state s with $x = a$ and $y = b$
- Assume that s satisfies P , that is, assume $0 \leq b \leq m$
- Consider the state t obtained by executing a transition from s
- If $a < m$ then $t(y) = b - 1$, else $t(y) = b$
- Can we conclude that $0 \leq t(y) \leq m$?
- No! When $b = 0$, $t(y)$ is negative.
- The proof fails. In fact, P is not an inductive invariant of T !

Why did the proof fail?

Consider the state s with $x = 0$ and $y = 0$

- State s satisfies $P : 0 \leq y \leq m$
- Executing a transition from s leads to state t with $x = 1$ and $y = -1$
- State t does **not** satisfy P

However, the state s in above argument is **not reachable**!

Cause of failure: P does not capture the correlation between the state components x and y

Solution: Inductive Strengthening

- Consider property $Q : (0 \leq y \leq m) \ \& \ (x + y = m)$ (if $(x < m)$ then $\{ x := x + 1 ; y := y - 1 \}$)
- Property Q implies property P
- While P is not an inductive invariant, Q is!
- It follows that P is **invariant**, even if it is **not inductive**

Proving Inductive Invariants: Example 3

Consider a transition system T given by

- State variables $\text{int } x, y$; initially: $x := 0 ; y := m$ for some fixed $m > 0$
- Transition description given by $\text{if } (x < m) \text{ then } \{ x := x + 1 ; y := y - 1 \}$

Property $Q : (0 \leq y \leq m) \ \& \ (x + y = m)$

Base case: Consider initial state $(x := 0, y := m)$. Check that it satisfies Q

Inductive case:

- Consider an arbitrary state s with $x = a$ and $y = b$
- Assume s satisfies Q , that is, assume $0 \leq b \leq m$ and $a + b = m$
- Consider the state t obtained by executing a transition from s
- If $a < m$ then $t(x) = a + 1$ and $t(y) = b - 1$, else $t(x) = a$ and $t(y) = b$
- But if $a < m$, since $b = m - a$, then $b > 0$, and thus $b - 1 \geq 0$
- In either case, the condition $(0 \leq t(y) \leq m) \ \& \ (t(x) + t(y) = m)$ holds

Conclusion: Property Q is an **inductive invariant**

General Proof Rule for Proving Invariants

To establish that a property P is an invariant of transition system T ,

find an *inductive strengthening* of P : a property Q such that

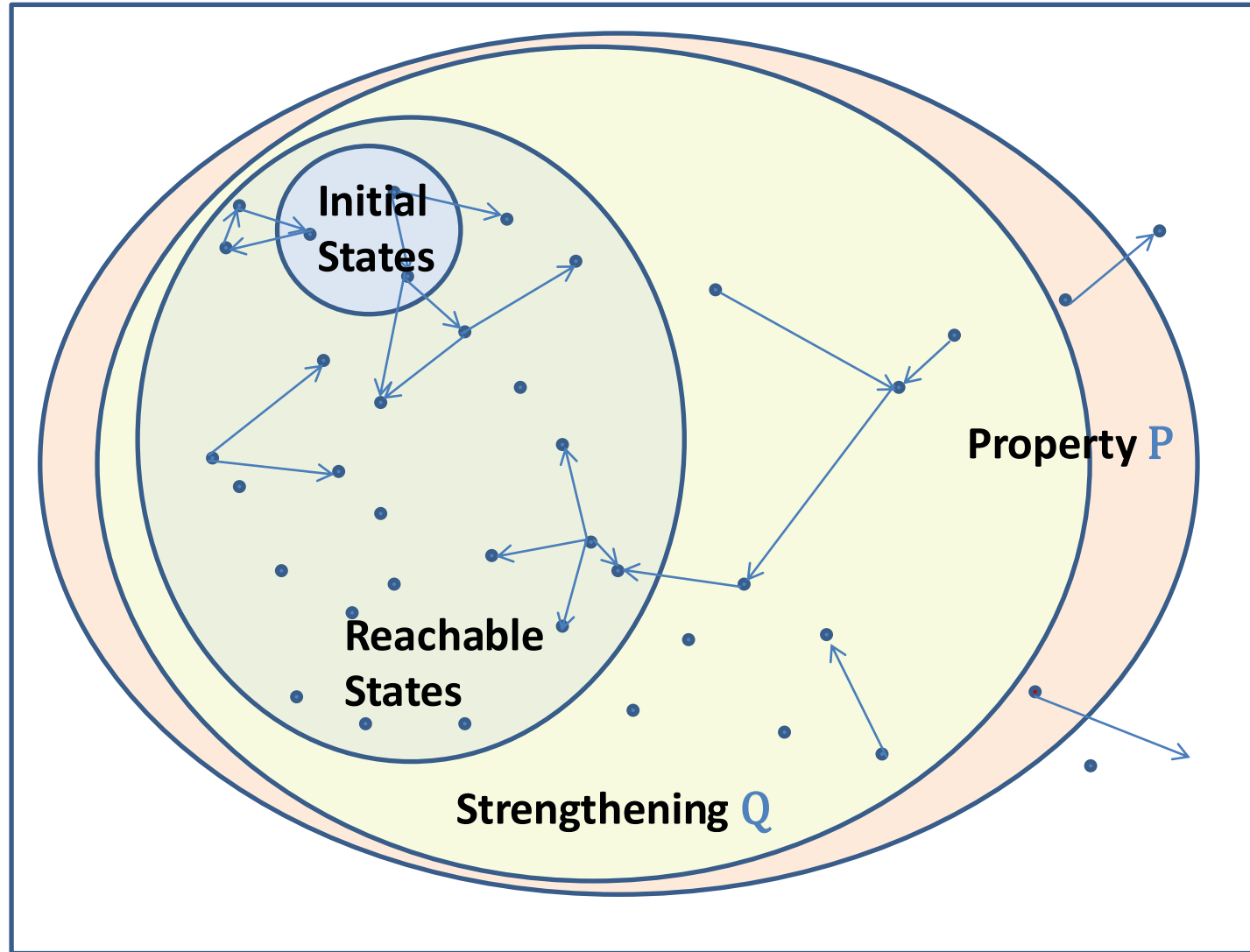
1. Q implies P (i.e., every state satisfying Q also satisfies P)
2. Q is an inductive invariant of T :
 - all initial states satisfy Q
 - For any states s, t such as s satisfies Q and $s \longrightarrow t$ is a transition, t satisfies Q

This is a *sound and complete* strategy for establishing invariants

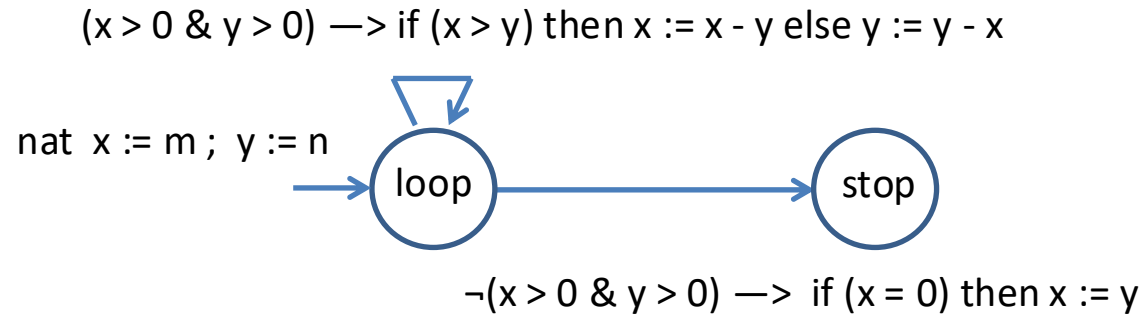
Sound: If P has an inductive strengthening Q , then P is invariant

Complete: If P is an invariant, then it has an inductive strengthening Q
(although Q may not be representable in the chosen property language)

Inductive Strengthening



Correctness of GCD



Property P : $\text{gcd}(x, y) = \text{gcd}(m, n)$

Verify that P is an inductive invariant (**Exercise**)

- Captures the core logic of the program: even though x and y are updated at every step, their gcd stays unchanged
- When switching to **stop**, if x is 0, then $\text{gcd}(x, y)$ is y ; if $y = 0$, then $\text{gcd}(x, y) = x$, and thus $x = \text{gcd}(m, n)$ upon switching to **stop**

Note: $(\text{mode} = \text{stop}) \Rightarrow (x = \text{gcd}(m, n))$ is **invariant**, but **not inductive**

Transition System for Leader Election

For each node n

Initial state:

$\text{int } id_n := n ; \text{ int } r_n := 1$

State transition update:

- Round counters r_n : $\text{if } r_n < N \text{ then } r_n := r_n + 1$
- Identifiers r_n : $id_n := \max \{ id_n, \max \{ id_m \mid m \text{ is connected to } n \} \}$

Invariants for Leader Election

Initial state:

- for all n , $\text{int } id_n := n$; $\text{int } r_n := 1$

State transition update: for all n ,

- if $r_n < N$ then $r_n := r_n + 1$
- $id_n := \max \{ id_n, \max \{ id_m \mid m \text{ is connected to } n \} \}$

Property: for all n , $id_n \geq n$

- Clearly an invariant; is it **inductive**?

Property: $id_1 \in ID$ with ID set of identifiers of all nodes

- **Not** an **inductive** invariant!
- During a transition $s \longrightarrow t$, value of id_1 in state t may equal value of id_m in state s , but property says nothing about $s(id_m)$
- What about: forall n , $id_n \in ID$? (**Exercise**)

Correctness of Leader Election

We expect id_n to be maximum of all identifiers after N rounds

Property: for all n , $(r_n = N) \Rightarrow (id_n = \max ID)$ (not inductive!)

Goal: Find inductive strengthening capturing correlation among all state variables at intermediate steps

Observe: for all nodes n , after k rounds, r_n is k and id_n is max of ids of nodes that are $< k$ hops away from n

Properties:

1. P_1 : for all m, n , $r_m = r_n$
2. P_2 : for all n , $id_n = \max \{ c \mid \text{distance}(c, n) < r_n \}$

Let's prove that $P_1 \& P_2$ is an inductive invariant!

Proof: Base Case

Initial state s : for each node n , $s(id_n) = n$ and $s(r_n) = 1$

Goal: Show that the following both hold in this initial state s

P_1 : for all m, n , $r_m = r_n$

P_2 : for all n , $id_n = \max \{ c \mid \text{distance}(c, n) < r_n \}$

P_1) $s(r_m) = s(r_n) = 1$; so P_1 holds

P_2) Consider a node n , we want to show

$$s(id_n) = \max \{ c \mid \text{distance}(c, n) < 1 \}$$

The only node c with $\text{distance}(c, n) < 1$ is n itself, and $s(id_n) = n$, so P_2 holds

Proof: Inductive Case

- Consider an arbitrary state s , and assume both P_1 and P_2 hold
- Let $s(r_n) = k$, for each node n
- For $k < N$, consider a successor state t of s
- **Goal:** Show that both P_1 and P_2 hold in state t
- To show P_1 , consider two nodes m and n
 - $t(r_m) = s(r_m) + 1 = k + 1$, and similarly, $t(r_n) = k + 1$, so P_1 holds in t
- To show P_2 , consider a node n , we want to show

$$t(id_n) = \max \{ c \mid \text{distance}(c, n) < k+1 \}$$

- **Assumption 1** (from inductive hypothesis): for all m ,
 $s(id_m) = \max \{ c \mid \text{distance}(c, m) < k \}$
- **Assumption 2** (from the transition relation):
 $t(id_n) = \max \{ s(id_n), \max \{ s(id_c) \mid c \text{ is connected to } n \} \}$

Proof: Inductive Case (Continued)

- Let $h = \max \{ c \mid \text{distance}(c, n) < k + 1 \}$ and $d = \text{distance}(h, n)$
- **Goal:** show that $t(\text{id}_n) = h$
- Since $d < k + 1$, either $d < k$ or $d = k$

Case ($d < k$):

- By Assumption 1, $s(\text{id}_n)$ cannot be $< h$, so must be h
- By Assumption 2, $t(\text{id}_n)$ cannot be $< s(\text{id}_n)$, so must be h

Case ($d = k$) :

- By basic properties of graphs, there must be a node m such that $\text{distance}(h, m) = k - 1$ and m is linked to n
- By Assumption 1, $s(\text{id}_m)$ cannot be $< h$, so must be h
- By Assumption 2, $t(\text{id}_n)$ cannot be $< s(\text{id}_m)$, so must be h
- The proof is complete!

Summary of Invariants

General ways to formulate and prove safety properties of programs/models/systems

Inductive invariant:

- Holds in initial states
- Is preserved by every transition

Crucial observation: To be inductive, a property must capture relevant relationships among enough state variables

Benefits of inductive invariants:

- Correctness reasoning becomes local (one only needs to think about what happens in one step)
- Tools are available to check if a property is an inductive invariant

Area of active research: Can a tool discover them automatically?

Automated Invariant Verification



Can such a verifier exist?

If so, what is the computational complexity of the verification problem?

Solving Invariant Verification

There is no generally efficient procedure to solve the verification problem

Solution 1: Simulation-based analysis

- Simulate the model multiple times, and check that each state encountered on each execution satisfies desired safety property
- Useful, practical in real-world, but gives only partial guarantee (and is known to miss hard-to-find bugs)

Solution 2: Semi-automated formal proofs using inductive invariants

- Only partial tool support possible, so requires considerable effort
- **Recent successes:** verified microprocessor, web browser, JVM

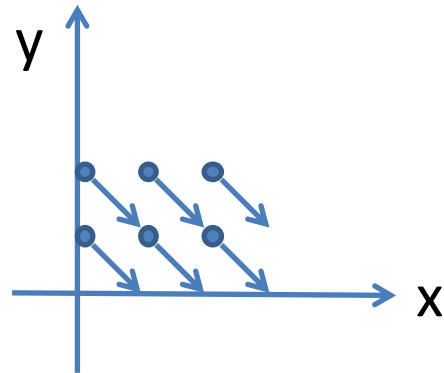
Solution 3: Exhaustive search through state-space

- Fully automated, but with scalability limitations (may not work!)
- Complementary to simulation, increasingly used in industry
- **Two approaches:** on-the-fly enumerative search, symbolic search

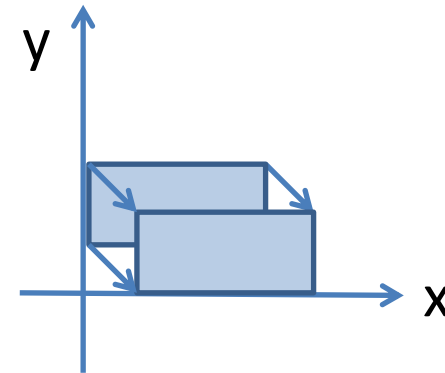
Computing Reachable States

Search algorithm can start with initial states, and explore transitions out of initial states systematically

Example: state vars are integers x, y ; we know that initially $0 \leq x \leq 2$ and $1 < y \leq 2$, and a single transition increments x and decrements y



Enumerative: Consider individual states



Symbolic: Consider set of states

Credits

Notes based on Chapter 3 of

Principles of Cyber-Physical Systems

by Rajeev Alur

MIT Press, 2015